

Atividade Prática 1 – Portas Lógicas

Pedro Lucas dos Reis Silva - RA: 2272040

27/03/2026

1 Introdução

Circuitos digitais combinacionais são aqueles cuja saída depende exclusivamente dos valores atuais das entradas, sem qualquer elemento de memória. As portas lógicas são os blocos fundamentais de construção desses circuitos, implementando as operações básicas da álgebra booleana.

Nesta atividade foram implementadas as seguintes portas lógicas em VHDL, com duas entradas (a e b) e uma saída independente para cada operação:

- **NOT** (inversora) – aplicada individualmente a cada entrada ($z1 = \text{NOT } a$, $z2 = \text{NOT } b$)
- **AND** – saída 1 somente quando ambas as entradas são 1
- **OR** – saída 1 quando pelo menos uma entrada é 1
- **NAND** – inverso do AND
- **NOR** – inverso do OR
- **XOR** – saída 1 quando as entradas são diferentes
- **XNOR** – saída 1 quando as entradas são iguais

1.1 Conceitos VHDL

A descrição em VHDL é composta por duas partes principais:

- **Entity**: define a interface externa do circuito (portas de entrada e saída), equivalente ao símbolo de um componente num esquemático.
- **Architecture**: define o comportamento interno do circuito. Nesta atividade foi utilizada a descrição no estilo *fluxo de dados* (dataflow), onde cada saída é descrita diretamente por sua equação booleana.

Todas as atribuições de sinal dentro da arquitetura ($\leq$) executam em **paralelo**, refletindo o comportamento real do hardware: todas as portas operam simultaneamente. O tipo `bit` foi utilizado por aceitar apenas '0' ou '1', suficiente para representar lógica digital básica.

Os códigos VHDL completos do circuito principal e do testbench encontram-se nos Apêndices A e B, respectivamente.

2 Tabela Verdade

A Tabela 1 apresenta os resultados esperados para cada combinação de entrada, cobrindo todas as 8 operações lógicas implementadas.

Tabela 1: Tabela verdade completa das 8 portas lógicas.

a	b	z1	z2	z3	z4	z5	z6	z7	z8
		NOT a	NOT b	AND	OR	NAND	NOR	XOR	XNOR
0	0	1	1	0	0	1	1	0	1
0	1	1	0	0	1	1	0	1	0
1	0	0	1	0	1	1	0	1	0
1	1	0	0	1	1	0	0	0	1

3 Simulação

A simulação foi realizada utilizando o GHDL (compilador e simulador VHDL open source) em ambiente Linux. O GHDL compilou o código VHDL, elaborou o circuito e executou a simulação gerando um arquivo de formas de onda no formato VCD (*Value Change Dump*). A visualização dos sinais foi feita com o GTKWave.

O testbench aplica as 4 combinações possíveis de **a** e **b** (00, 01, 10, 11), cada uma com duração de 1 ns. A Figura 1 apresenta as formas de onda obtidas, onde é possível verificar que os valores de cada saída correspondem exatamente à tabela verdade:

- **z1 e z2 (NOT)**: são o inverso de **a** e **b**, respectivamente.
- **z3 (AND)**: só vai para 1 quando **a**=1 e **b**=1 (instante 3 ns).
- **z4 (OR)**: vai para 1 sempre que ao menos uma entrada é 1.
- **z5 e z6 (NAND e NOR)**: são exatamente o complemento de **z3** e **z4**.
- **z7 (XOR)**: 1 quando as entradas diferem (instantes 1 e 2 ns).
- **z8 (XNOR)**: complemento do XOR, 1 quando as entradas são iguais.

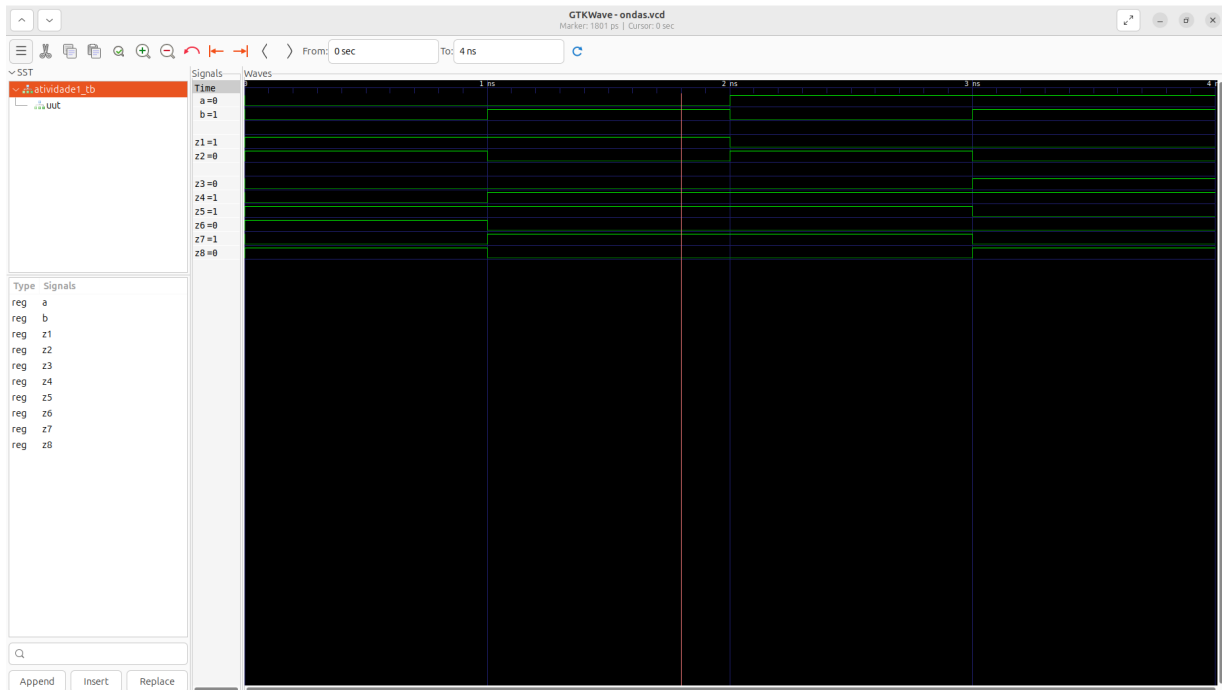


Figura 1: Formas de onda da simulação das 8 portas lógicas, visualizadas no GTKWave.

4 Diagrama RTL

O diagrama RTL (*Register Transfer Level*) representa a estrutura do circuito sintetizado, mostrando os componentes lógicos e suas interconexões. O código VHDL foi inserido no Quartus Prime e o diagrama foi gerado através da ferramenta *RTL Viewer* (Tools → Netlist Viewers → RTL Viewer).

A Figura 2 apresenta o diagrama obtido, onde se observa que:

- As entradas *a* e *b* alimentam diretamente todas as portas lógicas.
- Cada saída (*z1* a *z8*) está conectada à sua respectiva porta lógica.
- O diagrama evidencia a natureza puramente combinacional do circuito, sem elementos de memória.

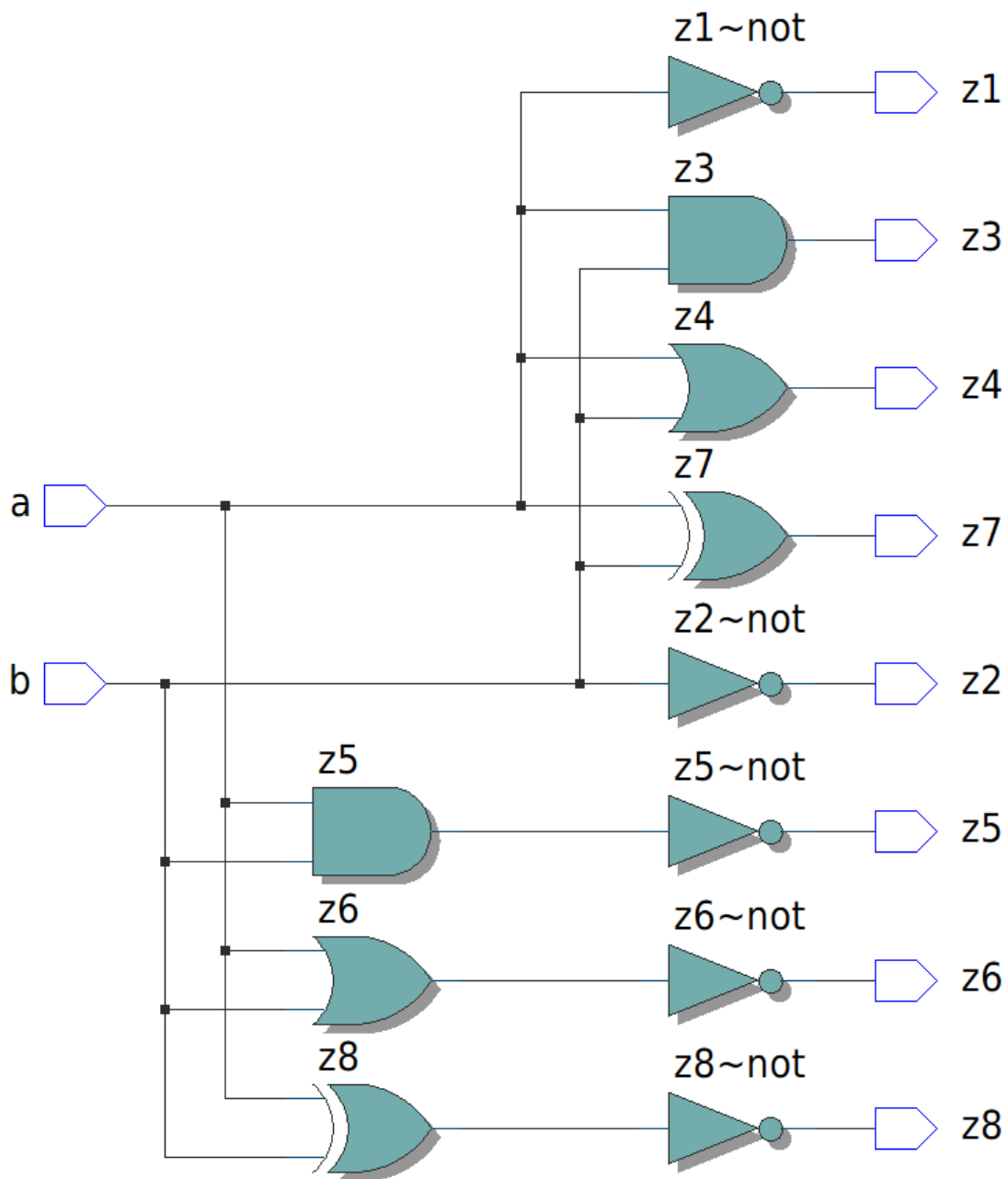


Figura 2: Diagrama RTL do circuito, gerado pelo Quartus Prime (RTL Viewer).

5 Ferramentas Utilizadas

O desenvolvimento desta atividade envolveu a utilização de diferentes ferramentas ao longo do processo:

- **GHDL** (v4.1.0) – compilador e simulador VHDL open source, utilizado para compilar o código e gerar as formas de onda em formato VCD.
- **GTKWave** (v3.3.116) – visualizador de formas de onda open source, utilizado para analisar os sinais da simulação.
- **Quartus Prime Lite** – ferramenta da Intel para síntese de circuitos em FPGA, utilizada para gerar o diagrama RTL do circuito.

- **VS Code** – editor de texto com extensão VHDL para syntax highlighting.

Durante o desenvolvimento, foram testadas alternativas open source para a geração do diagrama RTL, como o Yosys (sintetizador RTL) e o Logisim (simulador de circuitos digitais). O Yosys conseguiu gerar o diagrama a partir de uma descrição equivalente em Verilog, porém a visualização resultante não ficou tão intuitiva quanto a do Quartus. O Logisim permitiu desenhar o circuito manualmente com os símbolos das portas, mas não realiza síntese a partir de VHDL.

Concluiu-se que, em princípio, é possível realizar todo o fluxo de simulação e visualização utilizando exclusivamente ferramentas open source (GHDL + GTKWave + Yosys), reservando o Quartus apenas para a etapa final de síntese e gravação na placa FPGA. No entanto, essa abordagem requer um estudo mais aprofundado da integração entre as ferramentas, em especial o plugin ghdl-yosys para leitura direta de VHDL no sintetizador.

6 Considerações Finais

A atividade demonstrou o funcionamento correto das 8 portas lógicas implementadas em VHDL. Através da simulação com GHDL e GTKWave, verificou-se que os resultados obtidos coincidem com os valores esperados pela tabela verdade de cada operação.

A descrição em estilo *dataflow* mostrou-se adequada para circuitos combinacionais simples, onde cada saída é definida por uma equação booleana direta. O diagrama RTL gerado pelo Quartus confirmou a estrutura esperada do circuito, com todas as portas lógicas conectadas diretamente às entradas a e b.

A Código VHDL – Circuito Principal

```

1  -- =====
2  -- Atividade 1 - Portas Logicas
3  -- Disciplina: Logica Reconfiguravel - UTFPR Apucarana
4  -- Descricao: Implementacao de 8 portas logicas com entradas A e B
5  -- =====
6  library ieee;
7  use ieee.std_logic_1164.all;
8  -- =====
9  entity atividade1 is
10     port (
11         a      : in  bit;    -- entrada A
12         b      : in  bit;    -- entrada B
13         z1     : out bit;    -- NOT a
14         z2     : out bit;    -- NOT b
15         z3     : out bit;    -- AND
16         z4     : out bit;    -- OR
17         z5     : out bit;    -- NAND
18         z6     : out bit;    -- NOR
19         z7     : out bit;    -- XOR
20         z8     : out bit;    -- XNOR
21     );
22 end entity;
```

```

23  -- =====
24  architecture atividade1 of atividade1 is
25  begin
26      z1 <= not a;          -- inversao de A
27      z2 <= not b;          -- inversao de B
28      z3 <= a and b;        -- AND: saida 1 somente quando A=1 e B=1
29      z4 <= a or b;         -- OR: saida 1 quando ao menos uma entrada e 1
30      z5 <= a nand b;       -- NAND: inverso do AND
31      z6 <= a nor b;        -- NOR: inverso do OR
32      z7 <= a xor b;        -- XOR: saida 1 quando as entradas sao
                             diferentes
33      z8 <= a xnor b;       -- XNOR: saida 1 quando as entradas sao iguais
34  end architecture;

```

Listing 1: Arquivo atividade1.vhd – implementação das 8 portas lógicas.

B Código VHDL – Testbench

```

1  -- Testbench para atividade1: todas as portas logicas
2  -- Aplica as 4 combinacoes possiveis de A e B
3
4  entity atividade1_tb is
5  end entity;
6
7  architecture tb of atividade1_tb is
8      signal a                      : bit;
9      signal b                      : bit;
10     signal z1, z2, z3, z4, z5, z6, z7, z8 : bit;
11 begin
12
13     -- Instancia o componente a ser testado (DUT)
14     uut: entity work.atividade1
15         port map(
16             a  => a,
17             b  => b,
18             z1 => z1,  -- NOT a
19             z2 => z2,  -- NOT b
20             z3 => z3,  -- AND
21             z4 => z4,  -- OR
22             z5 => z5,  -- NAND
23             z6 => z6,  -- NOR
24             z7 => z7,  -- XOR
25             z8 => z8   -- XNOR
26         );
27
28     -- Processo de estimulo: testa todas as combinacoes de entrada
29     process
30     begin
31         a <= '0'; b <= '0'; wait for 1 ns;  -- combinacao 00
32         a <= '0'; b <= '1'; wait for 1 ns;  -- combinacao 01
33         a <= '1'; b <= '0'; wait for 1 ns;  -- combinacao 10
34         a <= '1'; b <= '1'; wait for 1 ns;  -- combinacao 11
35         wait;
36     end process;
37
38 end architecture;

```

Listing 2: Arquivo atividade1_tb.vhd – testbench para simulação.